

SYLLABUS JAVA PROGRAMMING

Paper Code: ETCS-307

L T/P C

Paper: Java Programming

3 1 4

INSTRUCTIONS TO PAPER SETTERS:

MAXIMUM MARKS: 75

1. Question No. 1 should be compulsory and cover the entire syllabus. This question should have objective or short answer type questions. It should be of 25 marks.

2. Apart from Question No. 1, rest of the paper shall consist of four units as per the syllabus. Every unit should have two questions. However, student may be asked to attempt only 1 question from each unit. Each question should be of 12.5 marks

Objective: To learn object oriented concepts and enhancing programming skills.

UNIT I

Overview and characteristics of Java, Java program Compilation and Execution Process Organization of the Java Virtual Machine, JVM as an interpreter and emulator, Instruction Set, class File Format, Verification, Class Area, Java Stack, Heap, Garbage Collection. Security Promises of the JVM, Security Architecture and Security Policy. Class loaders and security aspects, sandbox model [T1,R2][No. of Hrs.: 11]

UNIT II

Java Fundamentals, Data Types & Literals Variables, Wrapper Classes, Arrays, Arithmetic Operators, Logical Operators, Control of Flow, Classes and Instances, Class Member Modifiers Anonymous Inner Class Interfaces and Abstract Classes, inheritance, throw and throws clauses, user defined Exceptions, The String Buffer Class, tokenizer, applets, Life cycle of applet and Security concerns. [T1,T2][No. of Hrs.: 12]

UNIT III

Threads: Creating Threads, Thread Priority, Blocked States, Extending Thread Class, Runnable Interface, Starting Threads, Thread Synchronization, Synchronize Threads, Sync Code Block, Overriding Synced Methods, Thread Communication, wait, notify and notify all. AWT Components, Component Class, Container Class, Layout Manager Interface Default Layouts, Insets and Dimensions, Border Layout, Flow Layout, Grid Layout, Card Layout Grid Bag Layout AWT Events, Event Models, Listeners, Class Listener, Adapters, Action Event Methods Focus Event Key Event, Mouse Events, Window Event [T2][No. of Hrs.: 11]

UNIT IV

Input/Output Stream, Stream Filters, Buffered Streams, Data input and Output Stream, Print Stream Random Access File, JDBC (Database connectivity with MS-Access, Oracle, MS-SQL Server), Object serialization, Sockets, development of client Server applications, design of multithreaded server. Remote Method invocation, Java Native interfaces, Development of a JNI based application. Collection API Interfaces, Vector, stack, Hashtable classes, enumerations, set, List, Map, Iterators. [T1][R1][No. of Hrs.: 10]

Text Books:

[T1] Patrick Naughton and Herbertz Schidt, “Java-2 the complete Reference”,TMH

[T2] Sierra & bates, “Head First Java”, O’Reilly

Reference Books:

[R1] E. Balaguruswamy, “Programming with Java”, TMH

[R2] Horstmann, “Computing Concepts with Java 2 Essentials”, John Wiley.

[R3] Decker & Hirshfield, “Programming.Java”, Vikas Publication.

Introduction to Java

Java is a programming language created in 1995 in Sun Microsystem by James Gosling. Later, Sun Microsystem was acquired by Oracle in 2010. Since then, its a Oracle product.



Java is everywhere:

1. Java resides in mobile phones, client machines, server machines, embedded services, smart phones, cloud etc
2. It shares the same basic language features and libraries.
3. Principle of JAVA: Write Once, Run Anywhere (WORA)

Library?

It is a collection of pre defined classes. You can use these library either by inheriting them or instantiating them.

Java Editions

JAVA SE (STANDARD EDITION; CORE JAVA)

JAVA EE (ENTERPRISE EDITION: ADVANCE JAVA)

JAVA ME (MICRO EDITION FOR MOBILES)

Java Version

Version	Release date	End of Free Public Updates ^{[5][6]}	Extended Support Until
JDK Beta	1995	?	?
JDK 1.0	January 1996	?	?
JDK 1.1	February 1997	?	?
J2SE 1.2	December 1998	?	?
J2SE 1.3	May 2000	?	?
J2SE 1.4	February 2002	October 2008	February 2013
J2SE 5.0	September 2004	November 2009	April 2015
Java SE 6	December 2006	April 2013	December 2018
Java SE 7	July 2011	April 2015	July 2022
Java SE 8 (LTS)	March 2014	January 2019 for Oracle (commercial) December 2020 for Oracle (personal use) At least May 2026 for AdoptOpenJDK At least June 2023 ^[7] for Amazon Corretto	December 2030
Java SE 9	September 2017	March 2018 for OpenJDK	N/A
Java SE 10	March 2018	September 2018 for OpenJDK	N/A
Java SE 11 (LTS)	September 2018	At least August 2024 ^[7] for Amazon Corretto October 2024 for AdoptOpenJDK	September 2026
Java SE 12	March 2019	September 2019 for OpenJDK	N/A
Java SE 13	September 2019	March 2020 for OpenJDK	N/A
Java SE 14	March 2020	September 2020 for OpenJDK	N/A
Java SE 15	September 2020	March 2021 for OpenJDK	N/A
Java SE 16	March 2021	September 2021 for OpenJDK	N/A
Java SE 17 (LTS)	September 2021	TBA	TBA
Legend: ■ Old version ■ Older version, still maintained ■ Latest version ■ Future release			

Why Use Java?

- Java works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc.)
- It is one of the most popular programming language in the world
- It is easy to learn and simple to use
- It is open-source and free
- It is secure, fast and powerful
- It has a huge community support (tens of millions of developers)
- Java is an object oriented language which gives a clear structure to programs and allows code to be reused, lowering development costs
- As Java is close to [C++](#) and [C#](#), it makes it easy for programmers to switch to Java or vice versa

Features

- Simple
- Object Oriented
- Distributed
- Interpreted
- Robust
- Secure
- Portable
- Multi threaded
- Garbage Collector

Downloading & Installation of java

1. Jdk download

<https://www.oracle.com/in/java/technologies/javase-downloads.html>

← → ↻ oracle.com/in/java/technologies/javase-downloads.html

Oracle

Products Resources Support

Java / Technologies /
Java SE Downloads

Java SE Downloads

Java Platform, Standard Edition

Java SE 14

Java SE 14.0.2 is the latest release for the Java SE Platform

- [Documentation](#)
- [Installation Instructions](#)
- [Release Notes](#)
- [Oracle License](#)
 - [Binary License](#)
 - [Documentation License](#)

Oracle JDK

- [JDK Download](#)
- [Documentation Download](#)

2 <https://www.oracle.com/java/technologies/javase-jdk14-downloads.html>

← → ↻ oracle.com/java/technologies/javase-jdk14-downloads.html ☆

Oracle Products Resources Support [View Accounts](#)

Linux Debian Package	157.95 MB	jdk-14.0.2_linux-x64_bin.deb
Linux RPM Package	165.06 MB	jdk-14.0.2_linux-x64_bin.rpm
Linux Compressed Archive	182.06 MB	jdk-14.0.2_linux-x64_bin.tar.gz
macOS Installer	176.37 MB	jdk-14.0.2_osx-x64_bin.dmg
macOS Compressed Archive	176.79 MB	jdk-14.0.2_osx-x64_bin.tar.gz
Windows x64 Installer	162.11 MB	jdk-14.0.2_windows-x64_bin.exe
Windows x64 Compressed Archive	181.56 MB	jdk-14.0.2_windows-x64_bin.zip

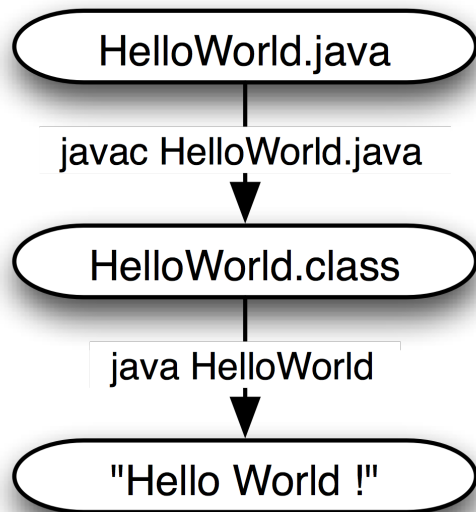
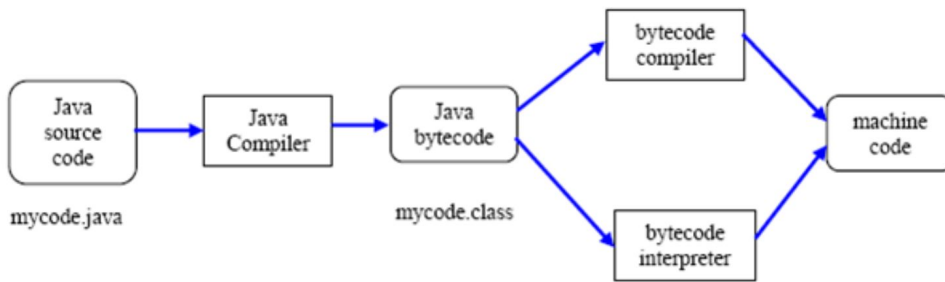
After Download and installation:

Setting a path:

1. C:\Program Files\Java\jdk-14.0.2\bin

2. Go to System settings of environment variable
3. In the Path, add: C:\Program Files\Java\jdk-14.0.2\bin
4. Add New:
5. Variable name: JAVA_HOME
6. Variable value : C:\Program Files\Java\jdk-14.0.2

How to Compile and Run?

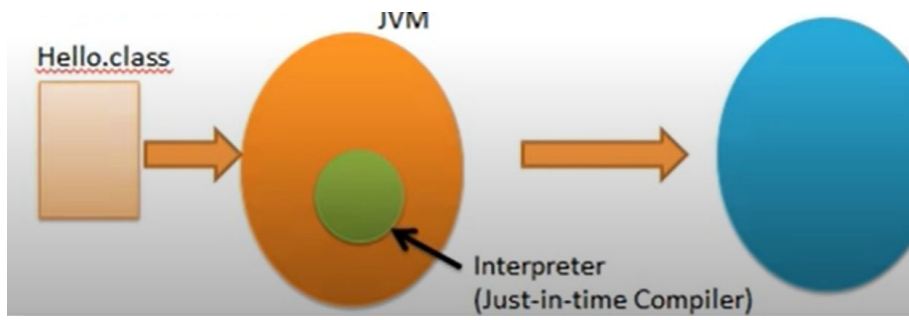


The language a computer can understand is called **machine language or machine code**.

When you run a program written in high-language, which is human-readable source code like *mycode.java*, you first need to compile it before you can run it. The compiler translates codes written in high-language into byte code (*mycode.class*)

There's a special interpreter that translates your byte code into machine language, this interpreter is called **Java Virtual Machine** a.k.a **JVM**. It is the JVM that runs your java code.

How to Run?



Difference between C and java?

C	JAVA
C is a Procedural Programming Language.	Java is Object-Oriented language.
C is a compiled language that is it converts the code into machine language so that it could be understood by the machine or system.	Java is an Interpreted language that is in Java, the code is first transformed into bytecode and that bytecode is then executed by the JVM (Java Virtual Machine).
C does not contain the property called Inheritance because it does not support OOPS, which is very useful for code reusability. Thus C is not suited when one has to relate the things according to the real world.	Java contains the property of Inheritance which is very useful in code reuseability.
Memory allocation can be done by malloc in C	Memory allocation can be done by a new keyword in Java.

Declaration of variables is at the beginning of the block but in the latest version of C that is C99 we can also declare variables anywhere.

We can declare variables anywhere.

free is used for freeing the memory in C.

A compiler will free up the memory internally by calling the garbage collector.

C does not supports Threading.

Java supports the concept of threading.

C supports pointers.

Java does not supports pointers.

It is not portable.

It is portable.

Call by value and call by reference is supported in C.

It only supports a call by value.

C is platform dependent.

Java is a platform independent.

It supports user-based memory management.	It internally manages the memory.
C is not robust that is strict type checking does not takes place while compile and run time.	Java is robust.
Exception handling cannot be directly achieved in C and thus it lacks the maintenance of normal flow of the program.	Exception Handling is supported in Java.
It follows a top-down approach.	Java follows a bottom-up approach.
Overloading functionality is not supported by C.	Java supports method overloading which helps in code readability.
C supports Preprocessors.	Java does not support Preprocessors.
C does not supports OOPS concept.	Java supports OOPS concept.

Union and structure datatypes are supported by C.	Java does not supports union and structures.
C supports the storage classes.	Whereas Java does not suport the storage classes.
Go-to statements are supported in C language.	Java does not supports go-to statements.
Virtual keywords are supported by C.	Virtual keywords are not supported by Java.
Overloading functionality is not supported by C.	Java supports method overloading which helps in code readability.
Default members of C are public.	Default members of Java are private.
Data hiding is done by using static in C.	Data hiding is done by using private in Java.

Difference between C++ and JAVA

Differences between Java and C++

PARAMETERS	JAVA	C++
Platform Dependency	Platform independent, Java bytecode works on any operating System.	Platform dependent, should be compiled for different platform.
Portability	It can run in any OS hence it is portable.	C++ is platform dependent hence it is not portable.
Compilation	Java is both Compiled and Interpreted Language.	C++ is only Compiled Language.

Memory Management	Memory Management is System Controlled.	Memory Management in C++ is Manual.
Virtual Keyword	It doesn't have Virtual Keyword.	It have Virtual Keyword.
Multiple Inheritance	It supports only single inheritance. Multiple inheritance achieved partially using interfaces.	It supports both single and multiple Inheritance.
Overloading	It supports only method overloading and doesn't allow operator overloading.	It supports both methods and operators overloading.
Pointers	It supports limited support for pointers.	It strongly supports Pointer.
Libraries	libraries have a wide range of classes for	C++ libraries are comparatively with low level functionalities.

	various high-level services	
Documentation Comment	It supports documentation comment (e.g, <code>/**.. */</code>) for source code.	It doesn't support documentation comment for source code.
Type	Java is only object-oriented programming language.	C++ is both a procedural and object-oriented programming language.
Input-Output mechanism	Java uses the (System class) System.in for input and System.out for output.	C++ uses cin for input and cout for an Output operation.
goto Keyword	Java doesn't support goto Keyword	C++ supports goto keyword.
Structures and Unions	Java doesn't support Structures and Unions.	C++ supports Structures and Unions.

Parameter Passing	Java supports only Pass by Value technique.	C++ supports both Pass by Value and pass by reference.
Global Scope	It supports no global scope.	It supports both global scope and namespace scope.
Object Management	Automatic object management with garbage collection.	It supports manual object management using new and delete.

JVM Architecture

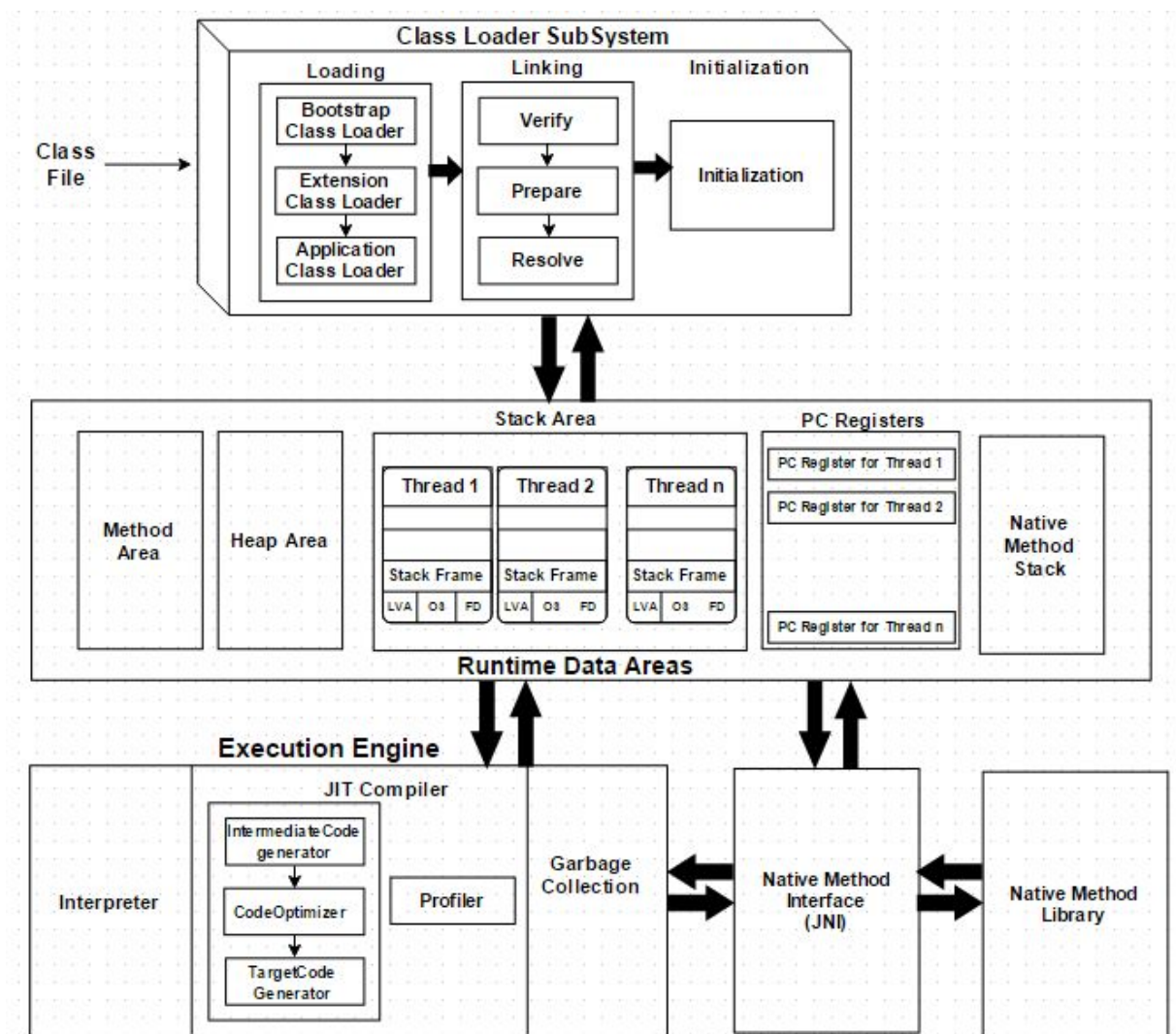
JVM is the heart of entire JAVA program execution process.

It is an abstract machine. It is specification that provides runtime environment in which JAVA bytecodes can be executed

Platform Dependent

Responsible for taking .class file and converting each byte code instructions into the machine language instructions that can be executed by the microprocessor.

Architecture



1. Class Loader : It is a sub system used for loading class files, it performs 3 main functions: Loading, linking and initialization

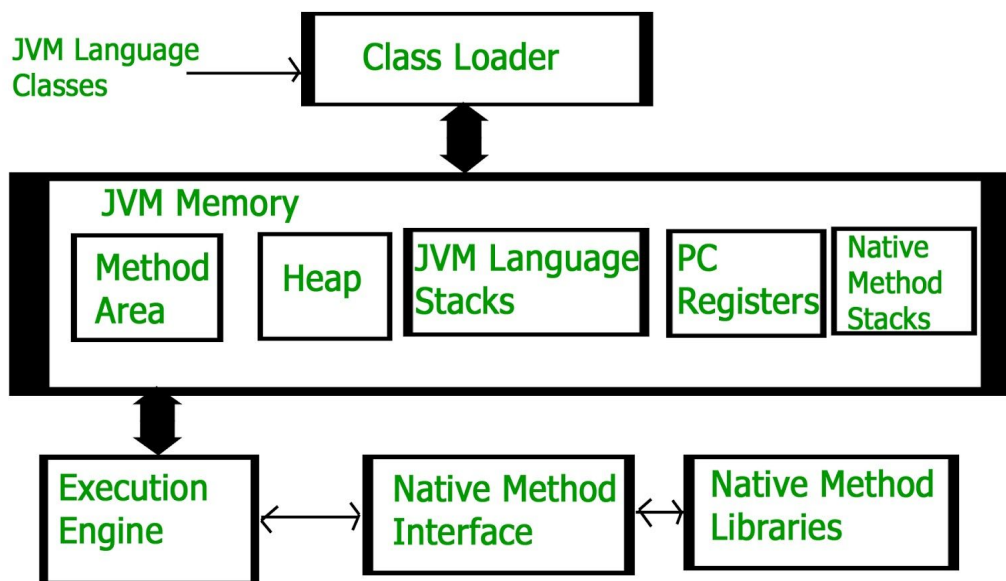
How JVM Works – JVM Architecture?

Last Updated: 21-05-2019

JVM(Java Virtual Machine) acts as a run-time engine to run Java applications. JVM is the one that actually calls the **main** method present in a java code. JVM is a part of JRE(Java Runtime Environment).

Java applications are called WORA (Write Once Run Anywhere). This means a programmer can develop Java code on one system and can expect it to run on any other Java enabled system without any adjustment. This is all possible because of JVM.

When we compile a .java file, .class files(contains byte-code) with the same class names present in .java file are generated by the Java compiler. This .class file goes into various steps when we run it. These steps together describe the whole JVM.



Class Loader Subsystem

It is mainly responsible for three activities.

- Loading
- Linking
- Initialization

Loading : The Class loader reads the *.class* file, generate the corresponding binary data and save it in method area. For each *.class* file, JVM stores following information in method area.

- Fully qualified name of the loaded class and its immediate parent class.
- Whether *.class* file is related to Class or Interface or Enum
- Modifier, Variables and Method information etc.

After loading *.class* file, JVM creates an object of type *Class* to represent this file in the heap memory. Please note that this object is of type *Class* predefined in *java.lang* package. This *Class* object can be used by the programmer for getting class level information like name of class, parent name, methods and variable information etc. To get this object reference we can use *getClass()* method of *Object* class.

Note : For every loaded *.class* file, only **one** object of *Class* is created.

```
Student s2 = new Student();  
// c2 will point to same object where  
// c1 is pointing  
Class c2 = s2.getClass();  
System.out.println(c1==c2); // true
```

Linking : Performs verification, preparation, and (optionally) resolution.

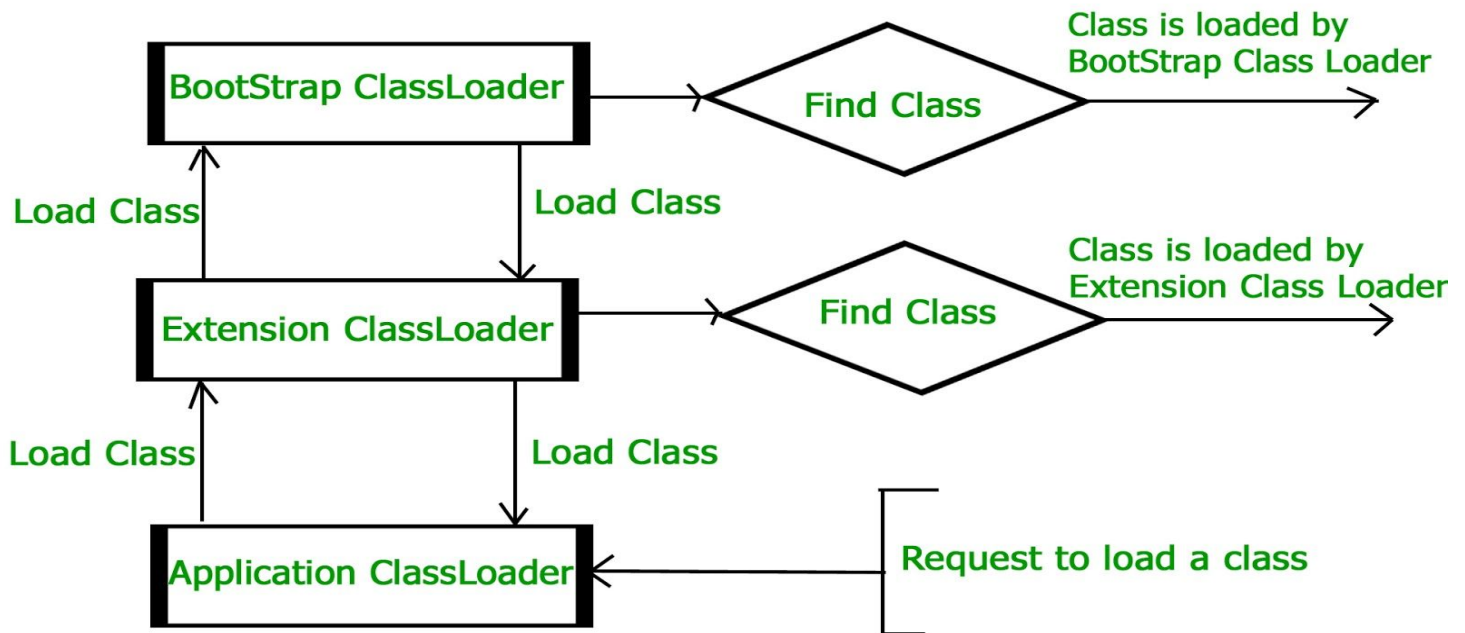
- *Verification* : It ensures the correctness of *.class* file i.e. it check whether this file is properly formatted and generated by valid compiler or not. If verification fails, we get run-time exception *java.lang.VerifyError*.
- *Preparation* : JVM allocates memory for class variables and initializing the memory to default values.
- *Resolution* : It is the process of replacing symbolic references from the type with direct references. It is done by searching into method area to locate the referenced entity.

Initialization : In this phase, all static variables are assigned with their values defined in the code and static block(if any). This is executed from top to bottom in a class and from parent to child in class hierarchy.

In general, there are three class loaders :

- *Bootstrap class loader* : Every JVM implementation must have a bootstrap class loader, capable of loading trusted classes. It loads core java API classes present in *JAVA_HOME/jre/lib* directory. This path is popularly known as bootstrap path. It is implemented in native languages like C, C++.
- *Extension class loader* : It is child of bootstrap class loader. It loads the classes present in the extensions directories *JAVA_HOME/jre/lib/ext*(Extension path) or any other directory specified by the *java.ext.dirs* system property. It is implemented in java by the *sun.misc.Launcher\$ExtClassLoader* class.
- *System/Application class loader* : It is child of extension class loader. It is responsible to load classes from application class path. It internally uses Environment Variable which mapped to *java.class.path*. It is also implemented in Java by the *sun.misc.Launcher\$AppClassLoader* class.

Note : JVM follow Delegation-Hierarchy principle to load classes. System class loader delegate load request to extension class loader and extension class loader delegate request to boot-strap class loader. If class found in boot-strap path, class is loaded otherwise request again transfers to extension class loader and then to system class loader. At last if system class loader fails to load class, then we get run-time exception *java.lang.ClassNotFoundException*.



JVM Memory

Method area : In method area, all class level information like class name, immediate parent class name, methods and variables information etc. are stored, including static variables. There is only one method area per JVM, and it is a shared resource.

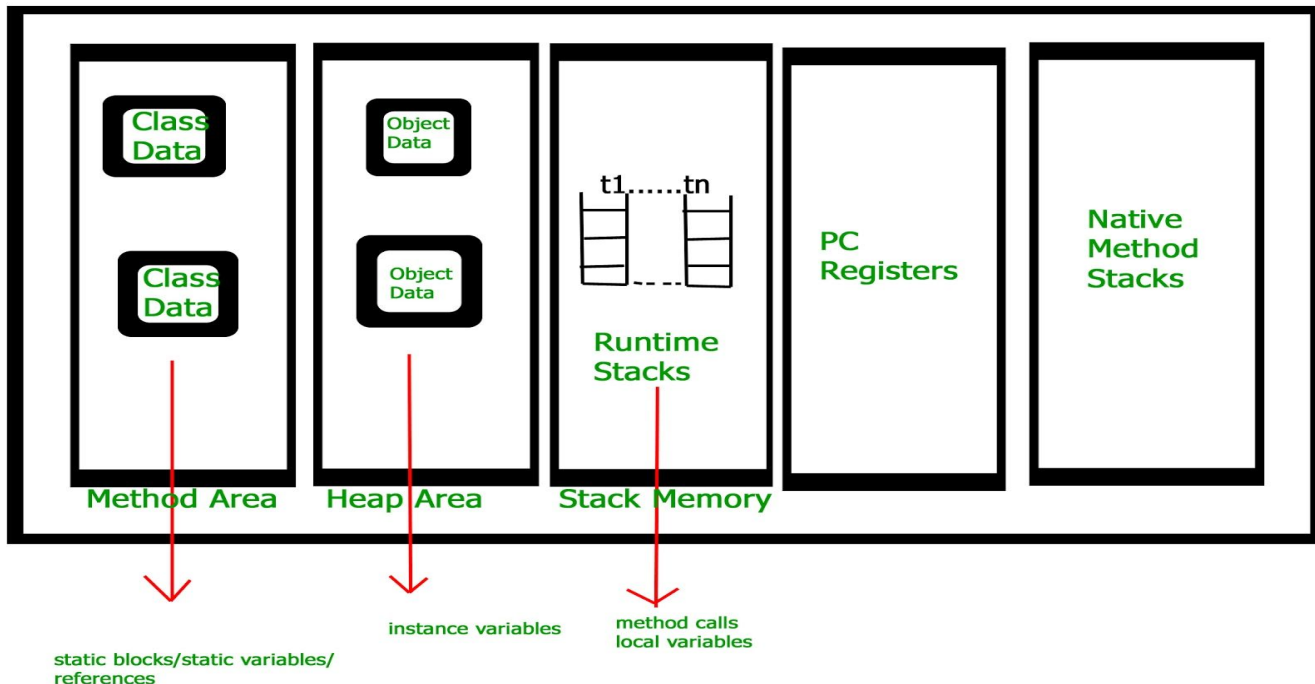
Heap area : Information of all objects is stored in heap area. There is also one Heap Area per JVM. It is also a shared resource.

Stack area : For every thread, JVM create one run-time stack which is stored here. Every block of this stack is called activation record/stack frame which store methods calls. All local variables of that

method are stored in their corresponding frame. After a thread terminate, it's run-time stack will be destroyed by JVM. It is not a shared resource.

PC Registers :Store address of current execution instruction of a thread. Obviously each thread has separate PC Registers.

Native method stacks :For every thread, separate native stack is created. It stores native method information.



Execution Engine

Execution engine execute the *.class* (bytecode). It reads the byte-code line by line, use data and information present in various memory area and execute instructions. It can be classified in three parts :-

- *Interpreter* : It interprets the bytecode line by line and then executes. The disadvantage here is that when one method is called multiple times, every time interpretation is required.
- *Just-In-Time Compiler(JIT)* : It is used to increase efficiency of interpreter. It compiles the entire bytecode and changes it to native code so whenever interpreter see repeated

method calls,JIT provide direct native code for that part so re-interpretation is not required,thus efficiency is improved.

- *Garbage Collector* : It destroy un-referenced objects.For more on Garbage Collector,refer [Garbage Collector](#).

Java Native Interface (JNI) :

It is an interface which interacts with the Native Method Libraries and provides the native libraries(C, C++) required for the execution. It enables JVM to call C/C++ libraries and to be called by C/C++ libraries which may be specific to hardware.

Native Method Libraries :

It is a collection of the Native Libraries(C, C++) which are required by the Execution Engine.